Computer Vision (4105109), Assignment #1

Camera projection matrix from 2D-3D point correspondences

Petlu Linu Zenas Paul, Student ID: 415410903

---

1. Method description

1.1 Data and notation

The input is 298 pairs of points, each with a pixel position (u, v) and a 3D position (X, Y, Z). I copied them from the Pt2D and Pt3D lists in the assignment PDF into one CSV file (columns u, v, X, Y, Z) and kept the order, so row i of the 2D list and row i of the 3D list describe the same point. Point numbers in this report are the row numbers of that file, counting from 1. The 3D points are not all in one plane (the smallest singular value of the centred points is 27% of the largest), which the DLT needs.

A pixel in homogeneous form is x = (u, v, 1) and a 3D point is X = (X, Y, Z, 1). The camera model is

$$x \sim P\,X$$

where P is a 3x4 matrix and "~" means equal up to a scale factor.

1.2 Projection matrix (task 1)

Let p_1, p_2, p_3 be the three rows of P. A point satisfies $u = (p_1 \cdot X) / (p_3 \cdot X)$ and $v = (p_2 \cdot X) / (p_3 \cdot X)$. Multiplying out gives two equations that are linear in the 12 entries of P:

$$p_1 \cdot X - u\,(p_3 \cdot X) = 0$$
$$p_2 \cdot X - v\,(p_3 \cdot X) = 0$$

If the 12 entries are written as one vector p, every point adds two rows to a matrix A:

$$\begin{bmatrix} X^T & 0^T & -u\,X^T \\ 0^T & X^T & -v\,X^T \end{bmatrix} p = 0$$

With 298 points A has 596 rows and 12 columns. The measurements are noisy, so A p = 0 has no exact solution. I minimise |A p| under the condition |p| = 1; the solution is the right singular vector of A that belongs to the smallest singular value (SVD).

Before building A I normalised both point sets, as described by Hartley and Zisserman [1]. The pixels are shifted so their centroid is at the origin and scaled so that their mean distance from the origin is √2. The 3D points are treated the same way with mean distance √3. This gives a 3x3 matrix T_2 and a 4x4 matrix T_3. The DLT is run on the normalised points and gives P~, which is converted back with

$$P = T_2^{-1}\,\tilde{P}\,T_3$$

Scale and sign of P are arbitrary. I divided P by the norm of (P_{31}, P_{32}, P_{33}), so this norm is 1, and picked the sign so that $p_3 \cdot X$ is positive for the points. With this choice $p_3 \cdot X$ is the depth of the point in front of the camera, in the units of the 3D data.

1.3 Calibration matrix, rotation and translation (task 2)

Write P = [M | p_4] where M is the left 3x3 block and p_4 the last column. If P = K [R | t] then M = K R and p_4 = K t. K is upper triangular and R is a rotation matrix, so K and R come from an RQ decomposition of M (scipy.linalg.rq). That decomposition is only unique up to signs: the signs of a column of K and of the matching row of R can be flipped together. I chose them so that the diagonal of K is positive. K is then divided by K_{33}, and P by the same number, so that K_{33} = 1. A rotation matrix must have det(R) = +1; the script stops with an error if it does not. Finally

$$t = K^{-1} p_4$$

I use the convention $X_{cam} = R\, X_{world} + t$. The camera centre in world coordinates is $C = -R^T t$. K has the form [[f_x, s, c_x], [0, f_y, c_y], [0, 0, 1]] with focal lengths f_x, f_y in pixels, skew s and principal point (c_x, c_y). The assignment asks for a "translation matrix"; I give the usual 3x1 vector t and, in addition, the camera centre C.

1.4 Projection and reprojection error (task 3)

Each 3D point is projected with P and divided by the third coordinate:

$$u' = (p_1 \cdot X) / (p_3 \cdot X), \quad v' = (p_2 \cdot X) / (p_3 \cdot X)$$

The code raises an error if the denominator is close to zero. The error of point i is the distance in pixels between the given and the projected point,

$$e_i = sqrt((u_i - u'_i)^2 + (v_i - v'_i)^2)$$

and the average projection error is the mean of e_i over all 298 points. I also report the median, maximum and RMS value.

The code is in six scripts that are run in order (see README.txt): step 2 does section 1.2, step 3 does 1.3 and step 5 does 1.4. Steps 1 and 4 only check the data and the decomposition, and step 6 contains the extra checks of section 2.4. Python 3 with NumPy, SciPy, pandas and matplotlib was used.

2. Experimental results

2.1 Estimated matrices

The matrices below are rounded to six decimals. The files in the results folder hold them at full precision.

```
P =
    666.738586     14.909862   -398.525915    -62.471700
     -0.256527   -663.940138   -288.672851      0.092784
     -0.000134      0.033682     -0.999433      0.020061

K =
    666.791420     -1.697070    398.712771
      0.000000    673.286513    266.146311
      0.000000      0.000000      1.000000
```

```
R =
     1.000000     -0.000323     -0.000145
    -0.000328     -0.999433     -0.033682
    -0.000134      0.033682     -0.999433

t =
    -0.105705     -0.007792      0.020061

C = -R^T t =
     0.105706     -0.008498      0.019772
```

Read as numbers: f_x = 666.79 px, f_y = 673.29 px, skew = −1.70 px, principal point (398.71, 266.15). The design matrix A has size 596 x 12. Its smallest singular value (normalised coordinates) is 0.068 and the next one is 7.18, so the solution is well defined.

2.2 Check of the decomposition

Quantity	Value
K is upper triangular, K_{33}	yes, 1
det(R)	1.000000
orthogonality error $\|R^TR - I\|$	9.2e-16
reconstruction error $\|P - s\,K[R\|t]\|$ (s = 1.0000)	1.6e-13
camera centre from $-M^{-1}p_4$ and from $-R^Tt$	same, to 6 decimals
rotation angle of R	178.07 degrees
viewing direction (third row of R)	(−0.000134, 0.033682, −0.999433)

2.3 Reprojection error

Quantity (pixels)	Value
average error	0.427162
median error	0.292053
maximum error (point 39)	4.456891
RMS error	0.640348
90th / 99th percentile	0.8908 / 1.8007
points with error above 1 px / above 2 px	21 / 2
average error without points 39 and 180	0.399981

All 298 points have a positive denominator, so all of them are in front of the camera. The five largest errors are:

Point	Error (px)	Given (u, v)	Projected (u, v)
39	4.4569	(556.045, 252.201)	(551.588, 252.159)
180	4.4429	(251.586, 494.888)	(255.934, 495.802)
14	1.9320	(527.842, 301.539)	(529.479, 300.514)
20	1.7967	(566.437, 126.101)	(568.230, 126.212)

Point	Error (px)	Given (u, v)	Projected (u, v)
78	1.5527	(297.908, 354.890)	(296.495, 354.247)

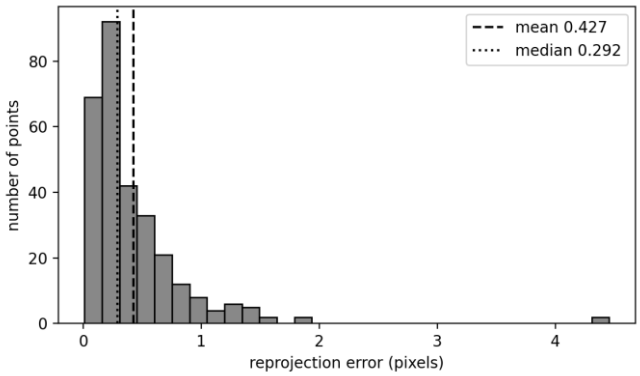


Figure 1. Histogram of the reprojection errors of the 298 points.

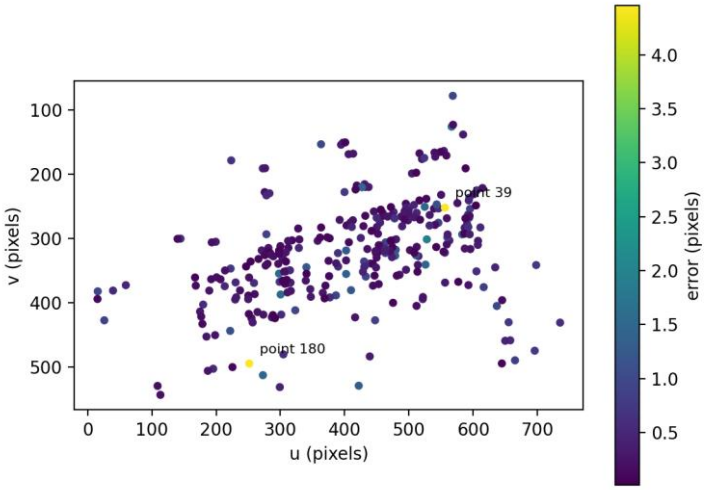


Figure 2. Given pixel positions, coloured by reprojection error. The vertical axis points down, as in an image.

2.4 Extra checks

These two checks are in step 6 and go beyond the three tasks. The first compares the DLT with and without the normalization step. In the second, P is estimated from four fifths of the points and the error is measured on the remaining fifth (5-fold hold-out, random split with a fixed seed).

Version	Average	Median	Maximum
DLT with normalization (all points)	0.427162	0.292053	4.456891
DLT without normalization (all points)	0.427089	0.285902	4.454302
5-fold hold-out (error on unseen points)	0.438037	0.293929	4.595125

Errors in pixels.

3. Discussion of results

The average error is 0.43 px and the median is 0.29 px, so for most points the projected position is within about half a pixel of the given one. The given points cover roughly 15 to 735 px in u

and 78 to 544 px in v, so this is small compared with the size of the point cloud in the image. Only 21 of 298 points are above 1 px.

The average is higher than the median because of two points. Points 39 and 180 have errors of about 4.4 px and together make up 32% of the total squared error; without them the average drops from 0.427 to 0.400. For point 39 the error is almost purely horizontal (the projected point is 4.46 px to the left of the given one and 0.04 px higher), for point 180 the projected point is 4.35 px to the right and 0.91 px lower. In Figure 2 the two points are far from each other, so they do not form a cluster. I do not know the reason from the data alone. A less accurate measurement of one of the coordinates of these two points would fit, but I did not test this.

The errors in section 2.3 are measured on the same points that were used to estimate P, so they show how well one matrix fits these points and nothing more. The hold-out check is a fairer test: the average error on unseen points is 0.438 px, close to 0.427 px, so P is not just memorising the points it was fitted to. This is one random split of the same 298 points, and all of them presumably come from one setup, so it does not say anything about other cameras or images.

Normalization made almost no difference here. The average error is 0.427162 px with it and 0.427089 px without it, and the maximum is also about the same. I therefore cannot claim from this data that normalization improved the accuracy. The reason to use it is that A is better conditioned, and the DLT minimises the algebraic error |A p|, not the pixel error, so a small difference in either direction is possible. Because of this, the later digits of P depend a little on the exact normalization, and P is only defined up to scale anyway. I did not try a nonlinear refinement of the pixel error.

R is close to a rotation of 180 degrees about the X axis (178.07 degrees in total, so about 1.9 degrees away from a pure flip). The viewing direction, the third row of R, is nearly (0, 0, −1). All Z values of the 3D points are between −6.4 and −3.6, so this means the camera looks along the negative Z direction towards the points, and the depths $p_3 \cdot X$ lie between 3.6 and 6.4 units. The camera centre is at about (0.106, −0.008, 0.020), close to the origin of the 3D coordinates. These values are consistent with each other, but I only have the numbers, not the real set-up, so I cannot say if they are the true camera position. The unit of t and C is the unit of the 3D data, which the assignment does not state.

The focal lengths differ by about 1% ($f_y/f_x = 1.0097$) and the skew is −1.70 px, which is small next to a focal length of about 670 px. There is no ground-truth calibration in the assignment, so I cannot tell whether these small deviations from square pixels and zero skew are real or only noise in the estimate.

The checks in section 2.2 show that the numbers from step 3 are consistent with each other: R is orthogonal with determinant 1, and K[R|t] gives P back. They are almost true by construction, because RQ always returns an orthogonal factor and t is calculated from P. They do not show that the camera model fits the data. The reprojection error is the part that does.

4. Problems or difficulties encountered

Reading the data. The point lists in the PDF are plain text, and in several places two points are written on one line. When copying them it is easy to lose the order or to pair a 2D point with the wrong 3D point. A wrong pairing would give errors of hundreds of pixels, so the small errors in section 2.3 are evidence that the pairing is right.

The SVD solution for P has arbitrary sign. I choose the sign so that the third-row dot products are positive for the supplied points, then apply consistent sign corrections during RQ decomposition to obtain a valid rotation matrix with determinant +1.

Non-unique RQ decomposition. The RQ decomposition can return K with negative entries on the diagonal. The signs of K and R have to be flipped together so that the product does not change, which is done in step 3.

Rounding in saved files. In an earlier version of the scripts the matrices were saved with 10 decimals. The decomposition checks then gave an orthogonality error of 7.9e-11 and a reconstruction error of 4.95e-8. These values came from the rounding in the files, not from the decomposition. The current scripts save with full precision, and the same checks give 9.2e-16 and 1.6e-13.

Two large errors. I could not find out why points 39 and 180 have errors of about 4.4 px (section 3).